

Continuous Integration and Continuous Deployment (CI/CD)

Introduction

Modern software development often involves multiple developers working on the same project simultaneously. As applications become larger and more complex, manually building, testing, and deploying software becomes time-consuming and error-prone. A small mistake introduced by one developer can affect the entire application if it is not detected early.

To solve these challenges, organizations use **Continuous Integration (CI)** and **Continuous Deployment/Continuous Delivery (CD)**, commonly referred to as **CI/CD**. CI/CD automates the software development lifecycle by ensuring that every code change is automatically tested, verified, and, if desired, deployed to production.

Today, CI/CD is a fundamental DevOps practice used by companies of all sizes, from startups to large technology organizations such as Google, Microsoft, Amazon, and Netflix.

What is CI/CD?

CI/CD is a collection of practices and automation tools that enable software to be developed, tested, and delivered efficiently.

It consists of two major components:

- **Continuous Integration (CI)** – Automatically builds and tests code whenever developers make changes.
- **Continuous Delivery/Continuous Deployment (CD)** – Automates the process of preparing or releasing software after it has successfully passed all tests.

Together, these practices reduce manual work, improve software quality, and enable faster delivery of new features.

Continuous Integration (CI)

Continuous Integration is the practice of frequently integrating code changes into a shared repository.

Instead of developers working separately for weeks before combining their work, each developer regularly pushes their changes to a version control system such as GitHub.

Whenever new code is pushed:

1. The latest code is downloaded.
2. Dependencies are installed.
3. The project is built.
4. Automated tests are executed.
5. The build status is reported.

If any test fails, developers are immediately notified so the issue can be fixed before additional work is added.

Example

Suppose three developers are working on an e-commerce website.

- Developer A modifies the login system.
- Developer B updates the shopping cart.
- Developer C changes the payment page.

Each developer pushes their work to GitHub.

A CI pipeline automatically:

- Downloads the latest code
- Builds the application
- Runs unit tests
- Runs integration tests
- Reports success or failure

If Developer A accidentally breaks the login functionality, the pipeline immediately reports the failure, preventing broken code from being merged into the main project.

Continuous Delivery (CD)

Continuous Delivery extends Continuous Integration by automatically preparing the application for deployment after successful testing.

The deployment package is created automatically and is always ready for release.

However, deployment to production usually requires manual approval.

Typical Continuous Delivery process:

Code → Build → Test → Package → Manual Approval → Production

Continuous Deployment

Continuous Deployment goes one step further than Continuous Delivery.

Once all automated tests pass successfully, the application is deployed automatically without any manual approval.

Process:

Code → Build → Test → Deploy Automatically

This approach is commonly used in organizations with highly reliable automated testing systems.

How CI/CD Works

The CI/CD process follows a sequence of automated stages known as a **pipeline**.

Step 1 – Developer Writes Code

A developer creates a new feature or fixes a bug.

Example:

- Add user authentication
 - Fix search functionality
 - Improve UI design
-

Step 2 – Code is Pushed to GitHub

The developer commits the changes and pushes them to a GitHub repository.

Example:

```
git add .  
git commit -m "Added login validation"  
git push origin main
```

This push triggers the CI/CD pipeline.

Step 3 – Source Code Checkout

The pipeline downloads the latest version of the repository.

This ensures every build starts with the newest project files.

Step 4 – Install Dependencies

Required libraries and packages are installed automatically.

Examples:

Python:

```
pip install -r requirements.txt
```

Node.js:

```
npm install
```

.NET:

```
dotnet restore
```

Step 5 – Build the Project

The source code is compiled or prepared for execution.

Examples:

Python projects may simply validate imports.

.NET projects use:

```
dotnet build
```

Java projects may use:

```
mvn package
```

Step 6 – Run Automated Tests

Automated tests verify that the application still works correctly.

Common tests include:

- Unit Tests
- Integration Tests
- API Tests
- UI Tests
- Regression Tests

If even one critical test fails, the pipeline stops immediately.

Step 7 – Generate Build Artifact

If testing succeeds, the pipeline creates a deployable package called an **artifact**.

Examples:

- Docker Image
- ZIP package
- JAR file
- DLL files
- Published website

Artifacts are stored so they can later be deployed.

Step 8 – Deployment

Depending on the project configuration, deployment may be:

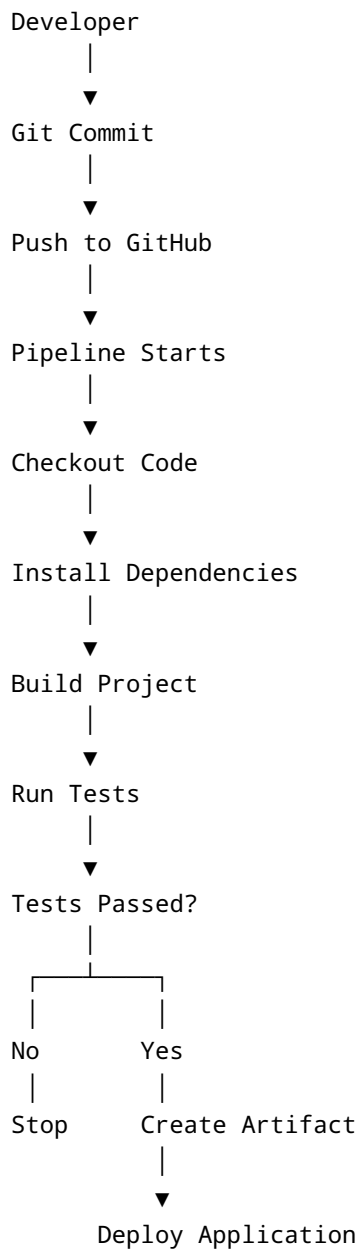
- Manual (Continuous Delivery)
- Automatic (Continuous Deployment)

Deployment targets may include:

- Cloud servers
- Docker containers
- Kubernetes clusters

- Virtual machines
 - Web hosting services
-

CI/CD Pipeline Overview



What Happens Without CI/CD?

Before CI/CD became common, software development relied heavily on manual processes.

Developers had to:

- Build applications manually.
- Execute tests by hand.
- Copy files to production servers manually.
- Verify deployments manually.

This approach introduced several problems:

1. Late Bug Detection

Errors were often discovered days or weeks after they were introduced, making them more difficult and expensive to fix.

2. Time-Consuming Testing

Every software update required developers or testers to manually execute test cases, increasing release times.

3. Frequent Human Errors

Manual deployment steps, such as copying incorrect files or forgetting configuration changes, could cause application failures.

4. Difficult Team Collaboration

When developers worked independently for long periods before merging code, integration conflicts became more common and resolving them required significant effort.

5. Slow Software Releases

Releasing a new version could take several days because building, testing, and deployment were performed manually.

6. Inconsistent Environments

Applications often behaved differently on development, testing, and production systems because setup steps were not automated.

Advantages of CI/CD

Implementing CI/CD provides significant benefits for software teams.

Faster Development

Automation reduces repetitive manual work, allowing developers to focus on implementing new features.

Early Bug Detection

Automated testing identifies problems immediately after code is committed.

Improved Software Quality

Every code change is validated using the same build and testing process, reducing the chance of introducing defects.

Faster Releases

Applications can be released multiple times a day instead of waiting for long release cycles.

Reduced Human Error

Automated pipelines perform builds, tests, and deployments consistently without relying on manual steps.

Better Team Collaboration

Frequent integration keeps the codebase synchronized, making collaboration between developers smoother.

Reliable Deployments

Every deployment follows the same verified process, reducing deployment failures.

Increased Productivity

Developers spend less time on repetitive tasks and more time building valuable features.

Popular CI/CD Tools

Several tools are widely used to automate CI/CD pipelines.

1. GitHub Actions

GitHub Actions is GitHub's built-in CI/CD platform. It allows developers to create automated workflows directly inside GitHub repositories using YAML configuration files. It integrates seamlessly with GitHub events such as code pushes, pull requests, issue creation, and scheduled tasks.

Advantages

- Built into GitHub
 - Easy workflow configuration
 - Supports Windows, Linux, and macOS runners
 - Large marketplace of reusable actions
 - Excellent for open-source and small-to-medium projects
-

2. Jenkins

Jenkins is one of the oldest and most popular open-source automation servers. It provides highly customizable CI/CD pipelines through plugins and supports virtually every programming language and deployment platform.

Advantages

- Open source
 - Thousands of plugins
 - Highly customizable
 - Suitable for enterprise environments
 - Supports distributed builds
-

3. GitLab CI/CD

Integrated directly into GitLab, GitLab CI/CD automates software building, testing, and deployment without requiring additional services.

Advantages

- Built into GitLab
 - Easy configuration
 - Strong DevOps integration
 - Container support
-

4. CircleCI

CircleCI is a cloud-based CI/CD platform focused on speed and scalability. It is commonly used for cloud-native and containerized applications.

Advantages

- Fast builds
 - Docker support
 - Parallel testing
 - Cloud-based infrastructure
-

5. Azure DevOps Pipelines

Azure DevOps Pipelines supports continuous integration and deployment for applications hosted on Microsoft Azure as well as other cloud providers.

Advantages

- Strong Microsoft ecosystem integration
 - Multi-platform support
 - Enterprise security
 - YAML pipelines
-

6. Travis CI

Travis CI is a hosted CI service that became popular among open-source projects due to its simple setup and GitHub integration.

Advantages

- Easy configuration
 - GitHub integration
 - Automatic builds
 - Good for open-source projects
-

GitHub Actions

What is GitHub Actions?

GitHub Actions is GitHub's native automation platform that enables developers to build CI/CD pipelines directly inside their repositories. It automates tasks whenever specific events occur, such as pushing code, opening a pull request, creating a release, or running on a schedule.

Workflows are defined in YAML files stored in:

```
.github/workflows/
```

Each workflow specifies:

- The event that triggers it.
- The operating system (runner) on which it will execute.
- The sequence of jobs and steps to perform.

Because workflows are stored alongside the project's source code, they are version-controlled, making it easy to review and update automation as the project evolves.

Working of GitHub Actions

The workflow begins when a configured event occurs, such as a developer pushing code to the repository.

Step 1 – Trigger Event

Examples of triggers include:

- Push to a branch
 - Pull request creation or update
 - Manual workflow execution
 - Scheduled execution (cron)
-

Step 2 – Runner Starts

GitHub provisions a runner, which is a virtual machine that executes the workflow. Runners are available for Linux, Windows, and macOS.

Step 3 – Checkout Repository

The runner downloads the latest version of the repository so that all subsequent tasks use the newest code.

Step 4 – Install Dependencies

The workflow installs the project's required packages, libraries, or SDKs.

Step 5 – Build the Application

The project is compiled or prepared for execution.

Step 6 – Execute Tests

Automated tests are run. If any required test fails, the workflow stops and reports the failure to the developers.

Step 7 – Optional Deployment

If configured, the workflow deploys the successfully tested application to a target environment such as a cloud server or hosting platform.

Benefits of GitHub Actions

- Native integration with GitHub repositories.
 - Minimal setup for GitHub-hosted projects.
 - Supports reusable actions from the GitHub Marketplace.
 - Provides detailed logs for each workflow run.
 - Automatically checks pull requests before merging.
 - Supports secret management for API keys and credentials.
 - Enables parallel execution of jobs to reduce build time.
-

Jenkins

What is Jenkins?

Jenkins is an open-source automation server designed to implement CI/CD pipelines. Unlike GitHub Actions, Jenkins is self-hosted, meaning organizations install and manage it on their own servers or virtual machines.

Jenkins is highly flexible because of its extensive plugin ecosystem, allowing it to integrate with source control systems, testing frameworks, cloud platforms, container technologies, and monitoring tools.

Working of Jenkins

Step 1 – Source Code Change

A developer pushes code to a source control repository such as GitHub.

Step 2 – Jenkins Trigger

Jenkins detects the change using webhooks or periodic polling and starts a pipeline.

Step 3 – Checkout Source Code

Jenkins retrieves the latest version of the project from the repository.

Step 4 – Build

The application is compiled or packaged according to the project's build configuration.

Step 5 – Automated Testing

Jenkins runs automated unit, integration, or UI tests. If any test fails, the pipeline stops and developers are notified.

Step 6 – Generate Artifact

If all tests succeed, Jenkins produces a deployable artifact such as a JAR file, Docker image, or published application package.

Step 7 – Deployment

Depending on the pipeline configuration, Jenkins deploys the artifact to a staging or production environment, either automatically or after manual approval.

Benefits of Jenkins

- Free and open source.
- Supports more than a thousand plugins.
- Works with almost every programming language and framework.
- Suitable for complex enterprise pipelines.
- Supports distributed builds using multiple build agents.
- Highly customizable for organization-specific workflows.
- Integrates with Docker, Kubernetes, cloud platforms, testing tools, and version control systems.

GitHub Actions vs Jenkins

Feature	GitHub Actions	Jenkins
Hosting	GitHub-hosted or self-hosted runners	Self-hosted automation server
Setup	Simple and quick	More complex
Configuration	YAML workflows	Jenkinsfile or UI
Integration	Native GitHub integration	Integrates with many platforms through plugins
Maintenance	Minimal for GitHub-hosted runners	Organization is responsible for server maintenance
Best For	GitHub-based projects, startups, and small-to-medium teams	Large organizations and highly customized enterprise pipelines

Conclusion

CI/CD has become an essential practice in modern software engineering because it automates the process of building, testing, and deploying software. By integrating code changes frequently and validating them through automated pipelines, development teams can detect errors early, improve software quality, and release new features more quickly.

Tools such as GitHub Actions and Jenkins provide reliable automation for implementing CI/CD pipelines. GitHub Actions offers a simple, repository-integrated solution that is ideal for GitHub-hosted projects, whereas Jenkins provides extensive customization and scalability for enterprise environments. Selecting the appropriate tool depends on factors such as project size, infrastructure requirements, team expertise, and deployment complexity.

Overall, adopting CI/CD leads to faster development cycles, more reliable releases, improved collaboration among developers, and a streamlined software delivery process.